

Providing State through the Router



Lara Newsom

Software Technical Leader | Speaker | Angular GDE

[lara-newsom](#) | www.linkedin.com



Route Properties to Provide Data

Title

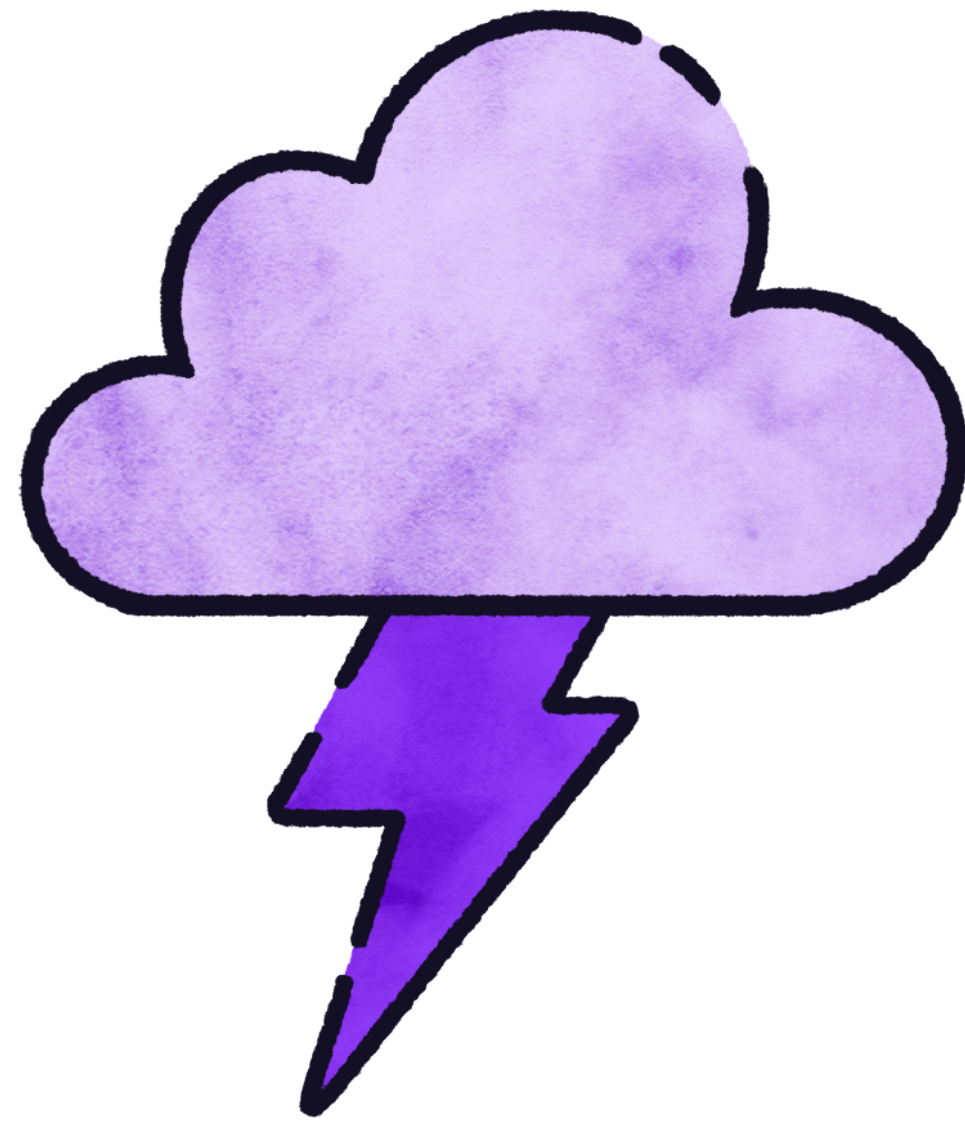
Data

Resolve

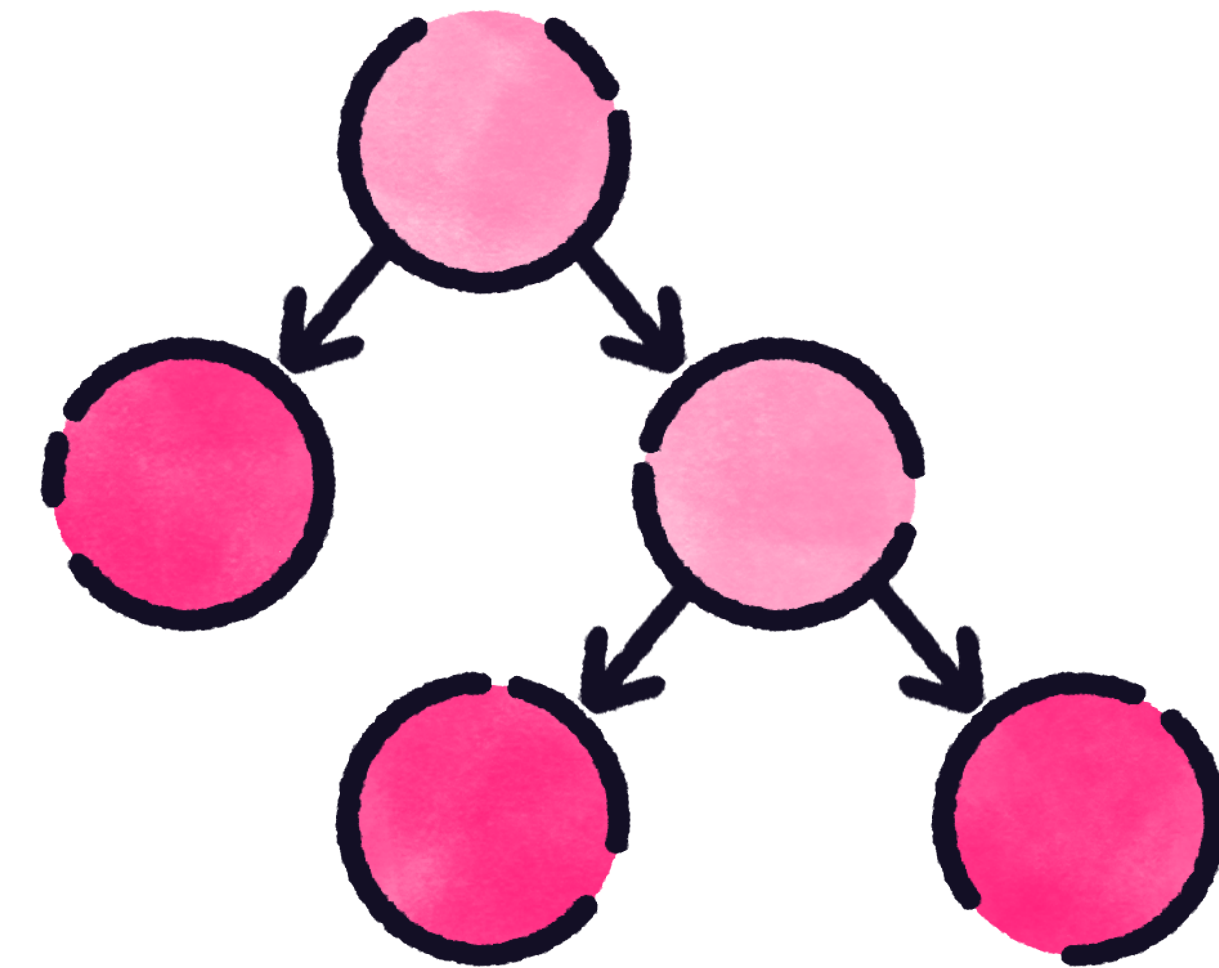
Providers



Accessing Data from a Route



The title, data, and resolve properties are accessible through the `ActivatedRoute`



The providers property uses Angular's dependency injection



Route Title Property

Sets the title element in the HTML head

Accessed through the `ActivatedRoute`'s title property

Provides the text for the browser tab

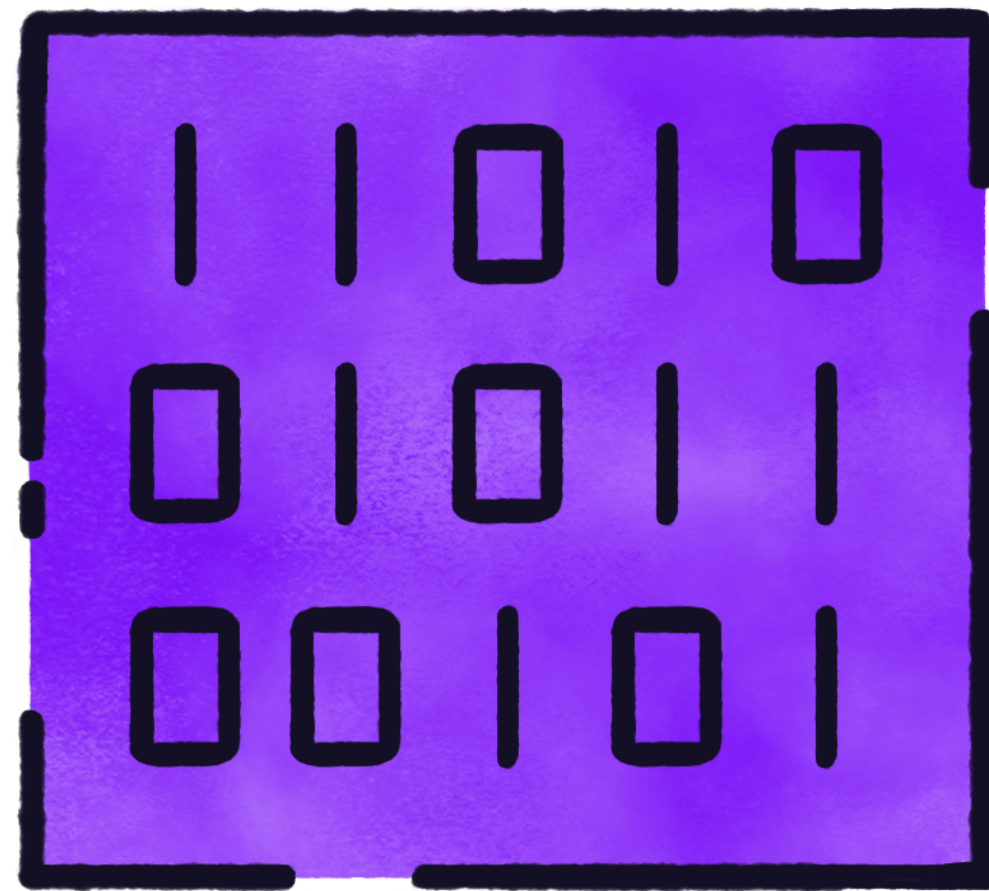
Improves search engine optimization

Sets the default bookmark title for a view

Provides important context for assistive technologies



Route Data Property



Accepts a static map of key-value pairs

Accessed through the `ActivatedRoute`'s `data` property

Component input binding can bind data to routed component inputs

Useful for passing more data than would be passed in the URL





Route Resolve Property

Used to pass dynamic state through the route

Maps a ResolveFn to a key on the ActivatedRoute's data property

Can return any kind of value including Observables, Promises, and RedirectCommands

Runs inside the injection context, so injectable dependencies can be used within the function



**Resolvers block navigation
until they complete.**



Route Providers Property

Allows providing route-level dependencies

Usually, services are provided in the root of the application, which makes them globally available

Dependencies can be scoped to a part of the application tree using component-level or route-level providers



Scoped Dependency Injection

Component-level Provider

Provided in the `ElementInjector`

Can't access global services

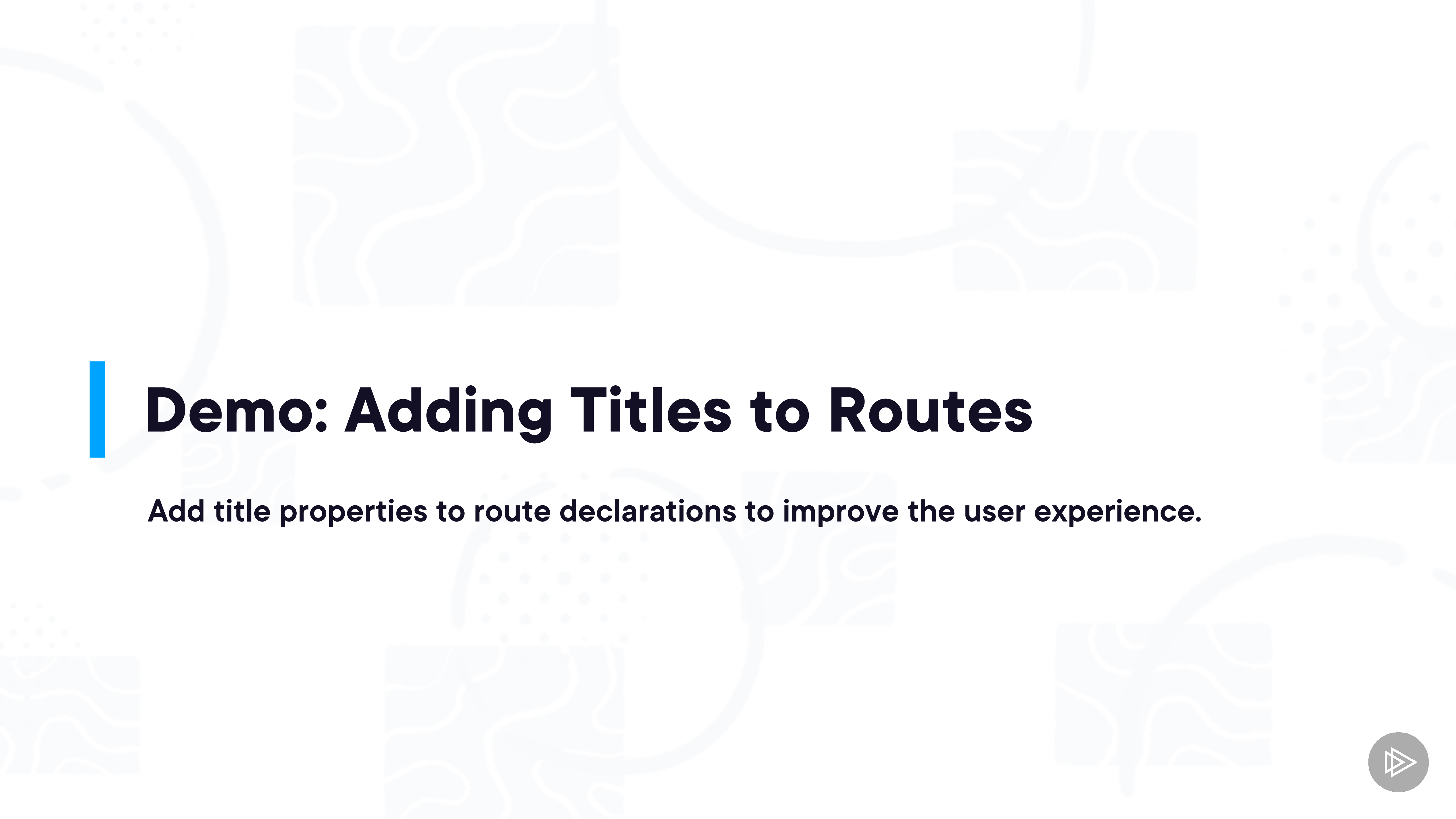
vs.

Route-level Provider

Provided in the `EnvironmentInjector`

Can access other global services





Demo: Adding Titles to Routes

Add title properties to route declarations to improve the user experience.



ResolveFn

```
MyResolver: ResolveFn = (  
  // Arguments provide access to route details  
  route: ActivatedRouteSnapshot, ←  
  state: RouterStateSnapshot ←  
) => {  
  // Dependencies can be injected  
  const myService = inject(MyService); ←  
  
  return myService.myTitle(); ←  
};
```



Generate a Resolver with the CLI

```
% ng generate resolver
```

```
✓ What name would you like to use for the  
resolver?
```

```
my-data
```

```
CREATE src/app/my-data.resolver.spec.ts
```

```
CREATE src/app/my-data.resolver.ts
```





Which Title Wins?

Titles can be declared on parent and child routes

Angular starts looking for a title on the active route and then walks up the route tree if none is found

The first title found wins

If there is no title, it falls back to the default in the index.html file



Demo: Providing Static Route Data



Reusable Components

There are different patterns for creating reusable components

Content projection is one option

Shared components can also receive data through the route data property



Demo: Providing Dynamic Route Data



Route Resolve Property

Defined using keys that are mapped to a `ResolveFn`

Functions can be declared inline or imported from other files

Can be generated using the command `ng generate resolver`

Useful for fetching required data before the component renders

Resolvers block navigation until they complete, so they should not be used for heavy or slow requests



Providing Route-level Dependencies

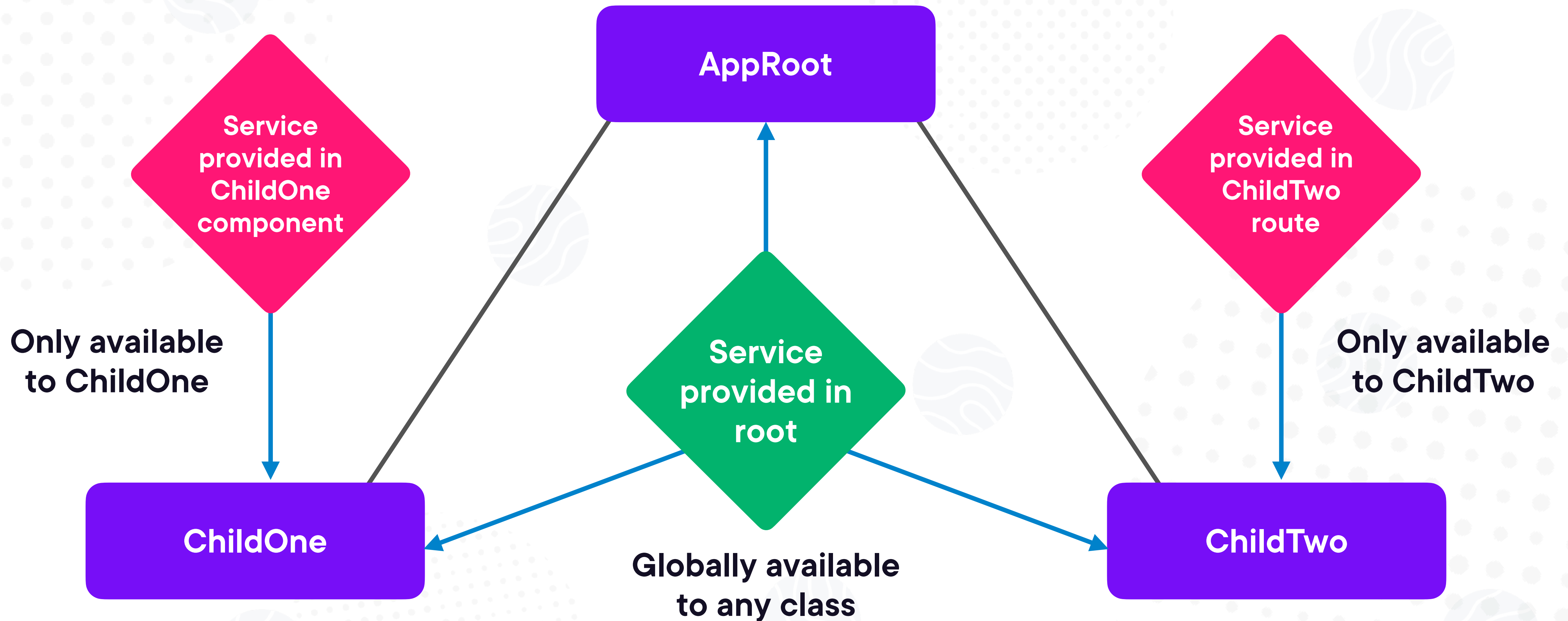


Injectable Services

```
@Injectable({ ←  
    providedIn: 'root' ←  
})  
export class MyService {  
    // State in this service is globally available  
    myData = signal<Data>({});  
  
    updateData(data: Data) {  
        this.myData.set(data);  
    }  
}
```



Providing Scoped Dependencies





Angular creates an injection token for each provided service

Tokens are used to retrieve instances of a dependency from the dependency injector

You can provide almost anything to the dependency injector, not just services



Angular View Transitions



Angular View Transitions

Users want fast, polished experiences on websites

The default-view transition creates a smooth fade between views

Improves the perceived performance of the application





Enabling View Transitions

Enabled using the `withViewTransitions` RouterFeature

Accepts an optional config property called `skipInitialTransition` to prevent slowing down the initial page load

Angular detects browser support and enables transitions only on compatible browsers

Custom transitions can be provided using CSS or an `onViewTransitionCreated` callback





Pluralsight

Angular Learning Path

pluralsight.com/paths/angular-development



Next Steps

Routing in server-side rendered applications

Testing Angular routes

Advanced implementations of lifecycle events

Custom route matching



Thank you for your time!

lara-newsom | [www.linkedin.com](https://www.linkedin.com/in/lara-newsom)

